Consultas con condiciones

Filtrar con `AND`, `OR`, `NOT`, `IN`, `LIKE` y compañía,
y presentar el resultado con `TOP`, `DISTINCT`, `ORDER BY` y `CASE`

Semana 08 - Sesión 15

Rel Guzman



Logro de la sesión

Al finalizar, podrás **combinar condiciones** con `AND`, `OR`, `NOT` y paréntesis, **filtrar** con `IN`, `BETWEEN`, `LIKE`, `EXISTS`, `ANY` y `ALL`, **tratar** los valores `NULL`, y **presentar** el resultado con `TOP`, `DISTINCT`, `ORDER BY` y `CASE`.

La tabla de ejemplo: la bodega creció

- Es la misma tabla `producto` de la sesión 14, con **dos filas más**.
- El **Chifles** no tiene categoría: su celda es *NULL*, es decir, **no se sabe**. Le daremos atención especial.
- Cada consulta parte de **estas seis filas**. Lo que la condición deja pasar se marca en amarillo.

`producto`

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20
6	Chifles	*NULL*	2.00	8

Ojo con esto

Los ejemplos están en MySQL. Casi todo se escribe **igual** en SQL Server y SQLite; las diferencias (`TOP`, algunos comodines, `ANY` y `ALL`) se señalan en su diapositiva.

Universidad Tecnológica del Perú

AND · OR · NOT

AND: se tienen que cumplir las dos

```
SELECT nombre, precio FROM producto
WHERE categoria = 'bebidas' AND precio < 3.50;
```

nombre	¿bebidas?	¿precio < 3.50?	¿sale?
Arroz	no	no	no
Gaseosa	**sí**	**sí**	**sí**
Agua	**sí**	**sí**	**sí**
Galletas	no	**sí**	no
Leche	**sí**	no	no
Chifles	¿?	**sí**	no

Una fila sale solo si **ambas** respuestas son **sí**. Basta **un no** para dejarla fuera.

¿? = la categoría del Chifles es `NULL`: no se sabe si es bebida.

¿Y el ¿? cuenta como resultado?

No. El `WHERE` solo deja pasar las filas cuya condición da **sí**. Un ¿? (desconocido) se trata igual que un **no**: el Chifles **no aparece** en el resultado del `SELECT`, y tampoco sale un error.

El resultado del AND

Tabla original

producto

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20
6	Chifles	*NULL*	2.00	8

Resultado del SELECT

```
SELECT nombre, precio FROM producto
WHERE categoria = 'bebidas' AND precio < 3.50;
```

nombre	precio
Gaseosa	3.00
Agua	1.50

`OR`: basta con que se cumpla una

```
SELECT nombre, categoria, stock FROM producto
WHERE categoria = 'snacks' OR stock = 0;
```

nombre	¿snacks?	¿stock = 0?	¿sale?
Arroz	no	no	no
Gaseosa	no	no	no
Agua	no	**sí**	**sí**
Galletas	**sí**	no	**sí**
Leche	no	no	no
Chifles	¿?	no	no

Una fila sale si **al menos una** respuesta es **sí**. Solo queda fuera cuando **todas** son no.

Si las dos se cumplen, la fila sale **una sola vez**.

En palabras simples

En la caja te dicen: «puedes pagar con **efectivo o** con **tarjeta**». Con cualquiera de los dos te atienden.

El resultado del `OR`

Tabla original

`producto`

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20
6	Chifles	*NULL*	2.00	8

Resultado del SELECT

```
SELECT nombre, categoria, stock FROM producto
WHERE categoria = 'snacks' OR stock = 0;
```

nombre	categoria	stock
Agua	bebidas	0
Galletas	snacks	50

NOT: da vuelta la respuesta

```
SELECT nombre, categoria FROM producto
WHERE NOT categoria = 'bebidas';
```

nombre	¿bebidas?	NOT: ¿sale?
Arroz	no	**sí**
Gaseosa	**sí**	no
Agua	**sí**	no
Galletas	no	**sí**
Leche	**sí**	no
Chifles	¿?	¿? no

- Lo que era **sí** pasa a **no**, y al revés.
- Lo que **no se sabe** sigue sin saberse: el Chifles **no sale**.
- Se combina con otros operadores: `NOT IN`, `NOT LIKE`, `NOT BETWEEN`, `NOT EXISTS`, `IS NOT NULL`.

Aquí `NOT categoria = 'bebidas'` equivale a `categoria <> 'bebidas'`. `NOT` brilla cuando niega una condición **compuesta**: `NOT (a OR b)`.

El resultado del NOT

Tabla original

producto

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20
6	Chifles	*NULL*	2.00	8

Resultado del SELECT

```
SELECT nombre, categoria FROM producto
WHERE NOT categoria = 'bebidas';
```

nombre	categoria
Arroz	abarrotes
Galletas	snacks

Paréntesis: primero AND, después OR

Sin paréntesis

```
WHERE categoria = 'bebidas'
   OR categoria = 'snacks'
   AND precio > 2;
```

El gestor lee: bebidas **o** (snacks **y** más de 2).

`Gaseosa, Agua, Leche`

sale el Agua, ¡que cuesta 1.50!

Con paréntesis

```
WHERE (categoria = 'bebidas'
   OR categoria = 'snacks')
   AND precio > 2;
```

Ahora: (bebidas **o** snacks) **y** más de 2.

`Gaseosa, Leche`

justo lo que se quería



orden en que el gestor los evalúa

En palabras simples

Es como en matemática: $2 + 3 \times 4 = 14$, pero $(2 + 3) \times 4 = 20$. `AND` es la multiplicación: va primero. **Si mezclas `AND` con `OR`, pon paréntesis siempre**, aunque no hagan falta.

Práctica 1 ¿Qué filas salen?

producto

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20
6	Chifles	*NULL*	2.00	8

Escribe los **nombres** que devuelve `SELECT nombre FROM producto` con cada condición.

Pista: las dos últimas solo se diferencian en los paréntesis. ¿Dan lo mismo?

Condición del `WHERE`	Resultado
`categoria = 'bebidas' AND stock > 10`	_____
`precio > 4 OR stock > 40`	_____
`NOT stock > 10`	_____
`(categoria = 'bebidas' OR categoria = 'snacks') AND stock < 15`	_____
`categoria = 'bebidas' OR categoria = 'snacks' AND stock < 15`	_____

Operadores

Los operadores lógicos, de más a menos usados

Operador	Pregunta que hace	Ejemplo	Uso
`AND`	¿se cumplen **las dos**?	`stock > 0 AND precio < 3`	●●●●●
`OR`	¿se cumple **alguna**?	`categoria = 'snacks' OR stock = 0`	●●●●●
`NOT`	¿**no** se cumple?	`NOT categoria = 'bebidas'`	●●●●○
`IN`	¿está en **esta lista**?	`categoria IN ('snacks', 'bebidas')`	●●●●○
`LIKE`	¿se **parece** a este patrón?	`nombre LIKE 'Ga%'`	●●●●○
`BETWEEN`	¿está en **este rango**?	`precio BETWEEN 1 AND 3`	●●●○○
`EXISTS`	¿hay **al menos una** fila?	`EXISTS (SELECT …)`	●●○○○
`ANY / SOME`	¿cumple con **alguno** de la lista?	`precio > ANY (SELECT …)`	●○○○○
`ALL`	¿cumple con **todos** los de la lista?	`precio > ALL (SELECT …)`	●○○○○

Todos responden **sí** o **no** para cada fila, así que se pueden mezclar con `AND`, `OR` y `NOT`. Los tres primeros ya los viste; `LIKE` tiene su propia sección.

`SOME` es **otro nombre** de `ANY`: hacen exactamente lo mismo.

IN y BETWEEN: listas y rangos

IN: ¿está en esta lista?

```
WHERE categoria IN ('snacks', 'abarrotes')
```

Arroz, Galletas

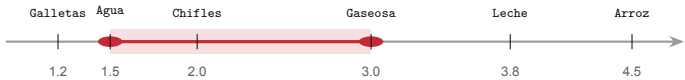
- Es una cadena de `OR` con `=`, pero **más corta**.
- Sirve con texto, números o fechas: `id IN (1, 3, 5)`.
- `NOT IN` pide lo contrario.

BETWEEN: ¿está en este rango?

```
WHERE precio BETWEEN 1.50 AND 3.00
```

Gaseosa, Agua, Chifles

- Es `precio >= 1.50 AND precio <= 3.00`: **incluye los extremos**.
- Primero el **menor**: `BETWEEN 3.00 AND 1.50` no trae nada.



Ejemplo con `IN` y `BETWEEN`

Tabla original

`producto`

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20
6	Chifles	*NULL*	2.00	8

Resultado del SELECT

```
SELECT nombre, categoria, precio FROM producto
WHERE categoria IN ('bebidas', 'snacks')
   AND precio BETWEEN 1.50 AND 3.00;
```

nombre	categoria	precio
Gaseosa	bebidas	3.00
Agua	bebidas	1.50

Las Galletas son `snacks`, pero cuestan 1.20: quedan **fuera del rango**. La Leche es bebida, pero cuesta 3.80.

Alias: apodos para columnas y tablas

Alias de columna

```
SELECT nombre AS articulo,
       precio AS costo
FROM producto;
```

articulo	costo
Arroz	4.50
Gaseosa	3.00
…	…

Cambia el **nombre que se muestra** en el resultado. La tabla no cambia.

Alias de tabla

```
SELECT p.nombre, p.precio
FROM producto p;
```

- `p` es un **apodo** de `producto`, válido solo en esta consulta.
- `p.nombre` se lee «la columna `nombre` de la tabla `p`».
- Sirve cuando una consulta usa **dos tablas**, como en `EXISTS`.

El `AS` es opcional: `producto AS p` y `producto p` son lo mismo.

`EXISTS`: ¿hay al menos una fila?

`venta`

id	producto_id	cantidad
1	1	5
2	2	3
3	4	10
4	2	1

cada fila es una venta de un producto

Pregunta: «¿qué productos **se vendieron** alguna vez?»

Para cada producto, `EXISTS` mira si en `venta` hay **al menos una fila** con su `id`:

- Si hay una o más: el producto **sale**.
- Si no hay ninguna: **no sale**.
- `NOT EXISTS` hace lo contrario.

En palabras simples

El bodeguero revisa sus boletas: «¿hay **al menos una** boleta de este producto?». No importa cuántas: con una basta.

Ejemplo con EXISTS

Tablas originales

producto

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20
6	Chifles	*NULL*	2.00	8

venta

id	producto_id	cantidad
1	1	5
2	2	3
3	4	10
4	2	1

Resultado del SELECT

```
SELECT p.nombre FROM producto p
WHERE EXISTS (SELECT * FROM venta v
              WHERE v.producto_id = p.id);
```

nombre
Arroz
Gaseosa
Galletas

Son los `id` 1, 2 y 4: los que aparecen en `venta`.

ANY, SOME y ALL: comparar con una lista

```
WHERE precio > ANY (SELECT precio FROM producto WHERE categoria = 'bebidas')
```

El SELECT de adentro da una **lista**: los precios de las bebidas, **3.00, 1.50 y 3.80**.

> ANY (o > SOME)

Mayor que **alguno** de la lista.
Basta con superar al **más bajo**: 1.50.

Arroz, Gaseosa, Leche, Chifles

> ALL

Mayor que **todos** los de la lista.
Hay que superar al **más alto**: 3.80.

Arroz

En palabras simples

«¿Corres más rápido que **alguno** del equipo?» Basta ganarle al más lento. «¿Más rápido que **todos**?» Hay que ganarle al más rápido.

SOME es otro nombre de ANY. Funcionan en MySQL y SQL Server; SQLite no los tiene.

Práctica 2 Elige el operador

Completa el espacio con `IN`, `BETWEEN`, `EXISTS` o `ALL`:

Pedido	Condición
Productos de snacks o de bebidas	`categoria _____ ('snacks', 'bebidas')`
Precio de 1 a 3 soles	`precio _____ 1 AND 3`
Productos que se vendieron	`_____ (SELECT * FROM venta …)`
Más caros que todas las bebidas	`precio > _____ (SELECT precio …)`

Pista: ¿es una **lista**, un **rango**, «**¿hay alguna?**» o «**contra todos**»?

Universidad Tecnológica del Perú

LIKE y comodines

LIKE: buscar por parecido

```sql
SELECT nombre FROM producto
WHERE nombre LIKE 'Ga%';   -- columna, LIKE, patrón
```

- = pide el texto **exacto**; LIKE pide que **se parezca** a un patrón.
- 'Ga%' se lee «empieza con **Ga** y sigue lo que sea».
- NOT LIKE se queda con los que **no** se parecen. En MySQL y SQL Server, normalmente **no distingue** mayúsculas: 'ga%' da lo mismo.

nombre
Gaseosa
Galletas

con `nombre = 'Ga'` no sale ninguno

Cómo se ve en la realidad

Cuando escribes *«gall»* en el buscador de una tienda en línea, detrás hay algo como `WHERE nombre LIKE '%gall%'`: trae todo lo que **contenga** esas letras.

Caracteres comodín: % y _

%

Reemplaza **cualquier cantidad** de caracteres: cero, uno o muchos.

_

Reemplaza **exactamente un** carácter, ni más ni menos.

Patrón	Se lee	Resultado
`'A%'`	empieza con A	`Arroz, Agua`
`'%s'`	termina en s	`Galletas, Chifles`
`'%ll%'`	contiene «ll» en cualquier parte	`Galletas`
`'_g%'`	la segunda letra es g	`Agua`
`'____'`	tiene exactamente 4 letras	`Agua`
`'L_che'`	L, una letra cualquiera, «che»	`Leche`

En palabras simples

Son como el **comodín** de las cartas: ocupan el lugar de lo que falta. El % es un comodín elástico, que se estira; el _ ocupa **una sola casilla**.

Patrones

A. ¿Qué nombres salen?

Condición	Resultado
`nombre LIKE 'G%'`	
`nombre LIKE '%a'`	
`nombre LIKE '%i%'`	
`nombre NOT LIKE '%a%'`	

B. Escribe el patrón

Pedido	Patrón
Termina en «oz»	
Tiene exactamente 5 letras	
Contiene «che»	
La tercera letra es u	

Los nombres son: `Arroz, Gaseosa, Agua, Galletas, Leche, Chifles`.

Pista: ¿el patrón habla del **comienzo**, del **final**, de **cualquier parte** o de una **posición exacta**?

Presentar el resultado

`TOP`: quedarse con las primeras filas

Pedido: «los **3 productos más caros**». Cada gestor lo escribe a su manera:

SQL Server

`TOP` va pegado al `SELECT`

```
SELECT TOP 3 nombre, precio FROM producto
ORDER BY precio DESC;
```

MySQL · SQLite

`LIMIT` va al final de todo

```
SELECT nombre, precio FROM producto
ORDER BY precio DESC LIMIT 3;
```

nombre	precio
Arroz	4.50
Leche	3.80
Gaseosa	3.00

Ojo con esto

Primero se ordena, después se cortan las 3 primeras. Sin `ORDER BY`, `TOP 3` trae «tres cualquiera»: no necesariamente las más caras.

SQL Server también acepta porcentajes: `SELECT TOP 50 PERCENT …` trae la mitad de las filas.

DISTINCT: sin repetidos

```sql
SELECT categoria FROM producto;
```

categoria
abarrotes
bebidas
bebidas
snacks
bebidas
NULL

6 filas, una por producto.

```sql
SELECT DISTINCT categoria FROM producto;
```

categoria
abarrotes
bebidas
snacks
NULL

4 filas: cada valor **una sola vez**.

- `DISTINCT` va **justo después** de `SELECT` y mira la **fila completa**: `SELECT DISTINCT categoria, precio` no quita nada, porque no hay dos filas con la misma pareja.
- `NULL` también cuenta como un valor: aparece una vez.

ORDER BY con varias columnas

```sql
SELECT categoria, nombre, precio
FROM producto
ORDER BY categoria, precio DESC;
```

categoria	nombre	precio
NULL	Chifles	2.00
abarrotes	Arroz	4.50
bebidas	Leche	3.80
bebidas	Gaseosa	3.00
bebidas	Agua	1.50
snacks	Galletas	1.20

en MySQL, SQL Server y SQLite, `NULL` va primero en orden `ASC`

- Ordena por la **primera** columna.
- La **segunda** solo decide cuando hay **empate** en la primera: las tres bebidas.
- Cada columna lleva su propio `ASC` o `DESC`.

En palabras simples

Es como la lista de alumnos: primero por **apellido**; si dos se apellidan igual, recién ahí se mira el **nombre**.

`IS NULL`: la casilla que nadie llenó

Así no

```
SELECT nombre FROM producto
WHERE categoria = NULL;
```

Ninguna fila, y sin error.

Así sí

```
SELECT nombre FROM producto
WHERE categoria IS NULL;
```

Sale el **Chifles**. Con `IS NOT NULL`, los otros cinco.

- `NULL` significa **«no se sabe»**. No es cero, ni un texto vacío `''`.
- Toda comparación con `NULL` da **desconocido**, y el `WHERE` solo deja pasar lo **verdadero**. Por eso el Chifles **no salió** con `NOT`: hay que pedirlo aparte.

```
WHERE categoria <> 'bebidas' OR categoria IS NULL;    -- Arroz, Galletas, Chifles
```

En palabras simples

En una ficha de inscripción, la casilla *teléfono* en blanco no quiere decir que tu número sea 0: quiere decir que **no lo sabemos**.

Ejemplo con `IS NULL`

Tabla original

`producto`

id	nombre	categoria	precio	stock
1	Arroz	abarrotes	4.50	30
2	Gaseosa	bebidas	3.00	12
3	Agua	bebidas	1.50	0
4	Galletas	snacks	1.20	50
5	Leche	bebidas	3.80	20
6	Chifles	*NULL*	2.00	8

Resultado del SELECT

```
SELECT nombre, categoria FROM producto
WHERE categoria IS NULL;
```

nombre	categoria
Chifles	*NULL*

CASE: una columna que decide

```sql
SELECT nombre, precio,
    CASE
        WHEN precio < 2 THEN 'barato'
        WHEN precio < 4 THEN 'regular'
        ELSE 'caro'
    END AS rango
FROM producto;
```

nombre	precio	rango
Arroz	4.50	caro
Gaseosa	3.00	regular
Agua	1.50	barato
Galletas	1.20	barato
Leche	3.80	regular
Chifles	2.00	regular

`CASE` va **dentro del** `SELECT`: crea una columna **solo en el resultado**, sin cambiar la tabla.

Gana el primero

El primer `WHEN` que se cumple decide. El Agua cumple los dos, pero queda «barato».

Ojo con el borde

El Chifles cuesta 2.00, que **no** es menor que 2: queda «regular».

Sin ELSE

Si nada se cumple y no hay `ELSE`, sale `NULL`: el Arroz quedaría sin rango.

Preguntas



Cierre: el DML en MySQL, SQL Server y SQLite

	MySQL	SQL Server	SQLite
`INSERT`, `SELECT`, `UPDATE`, `DELETE`	igual	igual	igual
Quedarse con 3 filas	`LIMIT 3`	`TOP 3`	`LIMIT 3`
Vaciar una tabla	`TRUNCATE TABLE`	`TRUNCATE TABLE`	`DELETE FROM`
Reemplazar un `NULL`	`IFNULL`	`ISNULL`	`IFNULL`
`ANY`, `SOME`, `ALL`	sí	sí	no
Comodín `[AG]` en `LIKE`	no	sí	no

Lo básico se escribe **igual** en los tres. Las diferencias están en los detalles; `COALESCE` funciona en todos.

Referencias

Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración* (9.ª ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos* (5.ª ed.). Addison Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.



UTP Universidad Tecnológica del Perú